

GraphQL Vulnerabilities vs REST API Vulnerabilities

Part 1: GraphQL Side (Focus: Query Logic Vulnerabilities)

1. Introspection Abuse (The “Map Maker”)

GraphQL contains a built-in feature called Introspection. This feature is designed to help developers understand the structure of the API. By using introspection, a client can ask the server questions about what data is available, what fields exist, what queries can be used, and how different objects are connected with each other.

An attacker performs this attack by sending a special hidden query to the GraphQL server. These queries usually begin with names such as `__schema` or `__type`. The server responds by providing a detailed description of the entire API structure.

The attacker can then study this information carefully. They can identify available objects, field names, hidden relationships, administrative functions, internal fields, and sensitive data structures. Even without access to the application source code, the attacker now understands how the backend is designed.

This attack works like receiving a complete map of a building before entering it. Instead of guessing where things are located, the attacker already knows the names of important fields and how to access them.

2. Depth/Circular DoS (The “Never-Ending Loop”)

GraphQL allows clients to request related data inside a single query. For example, a user may contain posts, and each post may contain an author, while that author again contains posts. This creates connected relationships between objects.

An attacker abuses this feature by creating an extremely deep nested query. The same relationships are repeated again and again inside the request. The attacker keeps asking for connected data repeatedly in a circular pattern.

The query may start by requesting a user. Inside that user, it requests posts. Inside those posts, it requests the author. Inside the author, it again requests posts. This pattern continues many times in the same request.

When the server receives this query, it tries to process every nested relationship step by step. It continuously performs database lookups and memory operations for each level of nesting. Because the request becomes extremely large internally, the server consumes huge amounts of CPU power and RAM while trying to complete the query.

The attacker does not need to send thousands of requests. A single deeply nested query is enough to force the server into excessive processing.

3. Alias/Batching Attack (The “Machine Gun”)

GraphQL supports a feature called Aliases. Aliases allow the same query to be executed multiple times inside one request while giving each result a different name. GraphQL also supports Batching, where multiple operations can be included together in a single HTTP request.

An attacker combines these features to perform many actions at once without sending multiple requests separately.

For example, instead of sending many login attempts one by one, the attacker sends only a single request. Inside that request, they include the login operation repeatedly using different aliases. Each alias contains a different password guess.

To the security system, it appears as only one HTTP request because everything is packed together inside a single request body. However, internally the server processes every individual operation separately.

The attacker can therefore perform many attempts simultaneously while appearing to send very little traffic. This allows multiple actions to happen in parallel within one request structure.

The attack works by hiding large numbers of operations inside a single request instead of distributing them across many visible requests.

Part 2: REST Side (Focus: Endpoint Design Vulnerabilities)

1. BOLA / IDOR (The “Number Guesser”)

REST APIs commonly use predictable URLs to access resources. For example, an order may be accessed using a path such as `/orders/12345`. Each number usually represents a specific record stored in the database.

An attacker first logs into their own account normally and observes how the URLs are structured. They notice that changing the number changes the resource being requested.

The attacker then manually modifies the number in the URL. Instead of requesting their own record, they request nearby records by increasing or decreasing the value. They continue changing the numbers repeatedly to access different records.

The server receives the modified request and searches for the requested record using the provided ID. If the backend only checks whether the user is logged in, but does not verify ownership of the resource, the server returns data belonging to other users.

The attacker can continue cycling through many IDs one after another and collect information from multiple accounts simply by changing numbers in the request.

This attack depends entirely on predictable identifiers and weak validation of ownership checks.

2. Mass Assignment (The “Hidden Field” Trick)

Modern backend frameworks often allow incoming JSON data to be automatically connected directly to database fields. This reduces development effort because developers do not need to manually assign every field one by one.

An attacker takes advantage of this automatic mapping process.

The attacker first interacts with a normal feature such as updating a profile. While the website only displays visible fields like name or email, the attacker assumes the database may also contain hidden fields such as account type, permissions, or internal settings.

Before the request reaches the server, the attacker intercepts the outgoing JSON data using browser developer tools or a proxy tool. They then manually add extra fields into the request that were never shown in the user interface.

The modified request is sent to the server containing both normal fields and attacker-added fields.

If the backend automatically accepts all incoming fields and saves them directly into the database, the hidden values are stored along with the legitimate data.

The attack succeeds because the server trusted all fields present in the incoming request without separating allowed fields from unexpected ones.

3. Excessive Data Exposure (The “TMI” Response)

In many REST applications, the backend sends large amounts of data to the frontend. Developers sometimes return complete database objects instead of carefully selecting only the required fields.

The website interface may display only a small portion of the information on the screen, such as a username or profile picture. However, the full API response often contains much more information in the background.

An attacker opens the browser’s Developer Tools and checks the Network tab. This allows them to inspect the raw JSON responses received from the server.

Inside the response data, the attacker may find additional fields that are not visible in the interface. These may include internal notes, addresses, phone numbers, hidden identifiers, account details, or other backend information.

The frontend may ignore or hide these fields visually, but the server has already delivered them to the client. The attacker simply reads the hidden data directly from the response body.

This attack occurs because the server sends more information than the frontend actually requires.